CPPopt_realtime2

June 16, 2026

[1]:
```
# ========================================
# SECTION 1 : IMPORT LIBRARIES
# ========================================

import pandas as pd
import numpy as np
from scipy.stats import pearsonr
from scipy.optimize import curve_fit
from collections import deque
from datetime import datetime, timedelta # Date and time handling
import matplotlib.pyplot as plt # Plotting
from IPython.display import clear_output # Clear previous output during live␣
 ↪monitoring
import time # Used for pause timings
import warnings # Suppress unnecessary warning

warnings.filterwarnings("ignore")

# ========================================
# SECTION 2 : LIVE DATA FILE
# ========================================

# This CSV file is continuously updated
# by the replay/writer notebook.
LIVE_FILE = "live_data.csv"

# ========================================
# SECTION 3 : SYSTEM PARAMETERS
# ========================================

# ------------------------------------------------
# DATA ACQUISITION SETTINGS
# Real ICU sampling interval.
# Meaning:
# One row in the original dataset represents
# approximately 5 seconds of real physiological data.
REAL_SAMPLE_INTERVAL = 5.0      # seconds
```

```
# Replay speed.
# During testing we do not want to wait
# 5 real seconds for every sample.
# Therefore:
# 1 dataset sample
# 0.05 seconds in replay mode
REPLAY_INTERVAL = 0.01          # seconds

# ------------------------------------------------
# PRx SETTINGS
# ------------------------------------------------

# PRx window size.
# PRx is calculated using the latest
# 60 samples of MAP and ICP.
# Since:
# 60 samples * 5 sec/sample
# 300 seconds
# 5 minutes
# Physiologically this corresponds to
# approximately a 5-minute PRx window.
PRX_WINDOW_SAMPLES = 60

# ------------------------------------------------
# CPPopt SETTINGS
# ------------------------------------------------

CPP_BIN_WIDTH = 5 # CPP Bin width = 5

CPP_MIN = 40        # Physiological CPP range accepted
CPP_MAX = 120       # for CPPopt.

# Number of historical PRx values
# used for CPPopt calculation.
# 240 PRx points approximately represent
# several hours of physiological behavior.
CPP_OPT_HISTORY = 240

# ------------------------------------------------
# CPPopt QUALITY CONTROL
# ------------------------------------------------

# Minimum acceptable R^2.
# If fitted quadratic curve is poor,
# CPPopt should not be trusted.
MIN_R2 = 0.20
```

```
# Minimum number of CPP bins required.
# Example:
# If only 2 bins exist,
# curve fitting becomes unreliable.
MIN_BINS_REQUIRED = 5

# Minimum number of PRx values inside each CPP bin.
# This prevents a bin from being
# created using only one random value.
MIN_VALUES_PER_BIN = 3

# Maximum acceptable PRx minimum. After fitting the U-curve,
# If minimum PRx itself is high autoregulation is poor and CPPopt is␣
 ↪physiologically meaningless.
MAX_ALLOWED_MIN_PRX = 0.25

# ------------------------------------------------
# GAP DETECTION SETTINGS
# ------------------------------------------------

# If timestamp difference exceeds this,
# a data gap is declared.
MAX_ALLOWED_GAP_SEC = 10

# ------------------------------------------------
# GRAPH DISPLAY SETTINGS
# ------------------------------------------------

# If TRUE:
# show entire graph history.
SHOW_ALL_GRAPH_POINTS = True

# If FALSE:
# show only latest N points.
GRAPH_POINTS_TO_SHOW = 100

# ========================================
# SECTION 4 : DATA BUFFERS
# ========================================

# ------------------------------------------------
# MAP BUFFER
# ------------------------------------------------

# Stores latest 60 MAP samples.
# deque(maxlen=60)
```

```
# automatically removes oldest value
# when a new value arrives.
map_buffer = deque(maxlen=PRX_WINDOW_SAMPLES)

# ------------------------------------------------
# ICP BUFFER
# ------------------------------------------------

# Stores latest 60 ICP samples.
icp_buffer = deque(maxlen=PRX_WINDOW_SAMPLES)

# ------------------------------------------------
# CPPopt HISTORY STORAGE
# ------------------------------------------------

# Stores historical PRx values.
# These values are used to generate
# CPPopt U-shaped curves.
prx_history = deque(maxlen=CPP_OPT_HISTORY)
mean_cpp_history = deque(maxlen=CPP_OPT_HISTORY)   # Stores corresponding mean␣
 ↪CPP values.
prx_time_history = deque(maxlen=CPP_OPT_HISTORY)    # Stores timestamps of PRx␣
 ↪calculations.
cppopt_time_history = deque(maxlen=CPP_OPT_HISTORY)  # Stores timestamps of␣
 ↪CPPopt calculations.

# ------------------------------------------------
# GRAPH DATA STORAGE
# ------------------------------------------------

cpp_series = []     # CPP graph values
prx_series = []     # PRx graph values
cppopt_series = []  # CPPopt graph values
time_series = []    # Common graph time axis

# ========================================
# SECTION 5 : STATUS VARIABLES
# ========================================

last_processed_row = 0            # Last processed CSV row prevents␣
 ↪reprocessing same rows.
last_timestamp = None             # Previous timestamp used for gap detection.
current_clinical_minute = None    # Current clinical minute used to aggregate␣
 ↪samples inside a minute.
samples_in_current_minute = 0     # Number of samples collected inside current␣
 ↪minute.
```

```
current_prx = np.nan                # Current PRx value.
current_cppopt = np.nan             # Current CPPopt value.

# Last valid CPPopt.
# During gaps we continue displaying
# this value instead of showing
# CPPopt unavailable.
last_valid_cppopt = np.nan

latest_map = np.nan    # Latest MAP value.
latest_icp = np.nan    # Latest ICP value
latest_cpp = np.nan    # Latest CPP value.

# Gap status flag.
# TRUE when replaying a detected gap.
gap_active = False

gap_reason = ""        # Gap reason text.

missing_samples = 0    # Number of missing samples corresponding to detected gap.
```

[2]:
```
# ========================================
# SECTION 6 : CPP CALCULATION
# ========================================

def calculate_cpp(map_value, icp_value):
    return map_value - icp_value  # CPP = MAP - ICP

# ========================================
# SECTION 7 : PRX CALCULATION
# ========================================

def calculate_prx(map_window, icp_window):
    # PRx requires a complete 60-sample window.
    if len(map_window) < PRX_WINDOW_SAMPLES:
        return np.nan, f"Need {PRX_WINDOW_SAMPLES - len(map_window)} more␣
 ↪samples"

    try:
        # Pearson correlation between MAP and ICP.
        prx, _ = pearsonr(map_window, icp_window)
        return prx, "Valid"
    except Exception as e:
        return np.nan, f"Correlation error : {str(e)}"

# ========================================
# SECTION 8 : QUADRATIC FUNCTION
```

```
# ========================================

def quadratic(x, a, b, c):
    # CPPopt assumes a U-shaped relationship:
    # PRx = a*x² + b*x + c
    # Minimum point of this curve
    # becomes CPPopt.
    return a*x**2 + b*x + c

# ========================================
# SECTION 9 : R² CALCULATION
# ========================================

def calculate_r2(y_actual, y_pred):
    ss_res = np.sum((y_actual - y_pred)**2)  # residual error
    ss_tot = np.sum((y_actual - np.mean(y_actual))**2)  # total variation

    if ss_tot == 0:
        return 0

    return 1 - (ss_res / ss_tot)

# ========================================
# SECTION 10 : CPPopt CALCULATION
# ========================================

def calculate_cppopt(prx_hist, cpp_hist):
    # ------------------------------------------------
    # STEP 1
    # Need enough historical PRx values.
    # ------------------------------------------------
    if len(prx_hist) < CPP_OPT_HISTORY:
        return (np.nan, f"Need {CPP_OPT_HISTORY - len(prx_hist)} more PRx␣
 ↪values")

    # ------------------------------------------------
    # STEP 2
    # Use latest CPP_OPT_HISTORY points.
    # ------------------------------------------------
    prx_hist = np.array(prx_hist)[-CPP_OPT_HISTORY:]
    cpp_hist = np.array(cpp_hist)[-CPP_OPT_HISTORY:]

    # ------------------------------------------------
    # STEP 3
    # Create CPP bins.
    # ------------------------------------------------
```

```
    bins = np.arange(cpp_hist.min(), cpp_hist.max() + CPP_BIN_WIDTH,␣
↪CPP_BIN_WIDTH)
    bin_centers = []
    bin_prx = []

    # ------------------------------------------------
    # STEP 4
    # Average PRx inside each CPP bin.
    # ------------------------------------------------
    for i in range(len(bins)-1):
        mask = ((cpp_hist >= bins[i]) & (cpp_hist < bins[i+1]))
        # Require minimum number of values per bin.
        if np.sum(mask) >= MIN_VALUES_PER_BIN:
            bin_centers.append((bins[i] + bins[i+1]) / 2)
            bin_prx.append(np.mean(prx_hist[mask]))

    # ------------------------------------------------
    # STEP 5
    # Require sufficient bins.
    # ------------------------------------------------
    if len(bin_centers) < MIN_BINS_REQUIRED:
        return (np.nan, f"Need at least {MIN_BINS_REQUIRED} CPP bins")

    bin_centers = np.array(bin_centers)
    bin_prx = np.array(bin_prx)

    try:
        # ------------------------------------------------
        # STEP 6
        # Fit quadratic curve.
        # ------------------------------------------------
        popt, _ = curve_fit(quadratic, bin_centers, bin_prx)
        a, b, c = popt

        # ------------------------------------------------
        # STEP 7
        # U-shape validation.
        # ------------------------------------------------
        if a <= 0:
            return (np.nan, "Quadratic coefficient a <= 0")

        # ------------------------------------------------
        # STEP 8
        # Calculate CPPopt.
        # ------------------------------------------------
        cppopt = -b / (2*a)
```

```
        # ------------------------------------------------
        # STEP 9
        # CPPopt physiological range.
        # ------------------------------------------------
        if cppopt < CPP_MIN or cppopt > CPP_MAX:
            return (np.nan, f"CPPopt outside range ({cppopt:.1f})")

        # ------------------------------------------------
        # STEP 10
        # Calculate fitted values.
        # ------------------------------------------------
        fitted = quadratic(bin_centers, *popt)

        # ------------------------------------------------
        # STEP 11
        # Calculate R².
        # ------------------------------------------------
        r2 = calculate_r2(bin_prx, fitted)
        if r2 < MIN_R2:
            return (np.nan, f"Poor fit R²={r2:.2f}")

        # ------------------------------------------------
        # STEP 12
        # Calculate minimum predicted PRx.
        # ------------------------------------------------
        min_prx = quadratic(cppopt, *popt)
        if min_prx > MAX_ALLOWED_MIN_PRX:
            return (np.nan, f"Minimum PRx too high ({min_prx:.2f})")

        # ------------------------------------------------
        # STEP 13
        # Success.
        # ------------------------------------------------
        return cppopt, "Valid"

    except Exception as e:
        return (np.nan, f"Curve fitting error : {str(e)}")

# ========================================
# SECTION 11 : GAP DETECTION
# ========================================

def check_gap(prev_time, current_time):
    if prev_time is None:
        return False, 0

    gap_sec = (current_time - prev_time).total_seconds()
```

```
    if gap_sec > MAX_ALLOWED_GAP_SEC:
        return True, gap_sec

    return False, gap_sec

# ========================================
# SECTION 11A : INSERT NaN POINTS
# ========================================

def insert_gap_nan_points(previous_timestamp, current_timestamp):
    global time_series
    global cpp_series
    global prx_series
    global cppopt_series

    if previous_timestamp is None:
        return

    gap_sec = (current_timestamp - previous_timestamp).total_seconds()
    missing_samples = int(gap_sec / REAL_SAMPLE_INTERVAL) - 1

    if missing_samples <= 0:
        return

    for i in range(1, missing_samples + 1):
        missing_time = previous_timestamp +␣
 ↪timedelta(seconds=i*REAL_SAMPLE_INTERVAL)
        time_series.append(missing_time)
        cpp_series.append(np.nan)  # break CPP graph
        prx_series.append(np.nan)  # break PRx graph
        cppopt_series.append(np.nan)  # break CPPopt graph
```

[3]:
```
# ========================================
# SECTION 12 : LIVE PLOTTING
# ========================================

def plot_live():
    # ----------------------------------------
    # Clear previous output so that
    # only latest graphs remain visible.
    # ----------------------------------------
    clear_output(wait=True)

    # ----------------------------------------
    # Select graph range.
    # ----------------------------------------
```

```
    if SHOW_ALL_GRAPH_POINTS:
        plot_time = time_series
        plot_cpp = cpp_series
        plot_prx = prx_series
        plot_cppopt = cppopt_series
    else:
        plot_time = time_series[-GRAPH_POINTS_TO_SHOW:]
        plot_cpp = cpp_series[-GRAPH_POINTS_TO_SHOW:]
        plot_prx = prx_series[-GRAPH_POINTS_TO_SHOW:]
        plot_cppopt = cppopt_series[-GRAPH_POINTS_TO_SHOW:]

    # ----------------------------------------
    # Create figure.
    # ----------------------------------------
    plt.figure(figsize=(15,10))

    # ========================================
    # CPP GRAPH
    # ========================================
    plt.subplot(3,1,1)
    plt.plot(plot_time, plot_cpp, linewidth=1.5)
    plt.title("Cerebral Perfusion Pressure (CPP)")
    plt.ylabel("CPP (mmHg)")
    plt.grid(True)

    # ----------------------------------------
    # Reference lines.
    # ----------------------------------------
    plt.axhline(y=CPP_MIN, linestyle="--", alpha=0.5)
    plt.axhline(y=CPP_MAX, linestyle="--", alpha=0.5)

    # ========================================
    # PRx GRAPH
    # ========================================
    plt.subplot(3,1,2)
    plt.plot(plot_time, plot_prx, linewidth=1.5)
    plt.title("Pressure Reactivity Index (PRx)")
    plt.ylabel("PRx")
    plt.grid(True)

    # ----------------------------------------
    # PRx reference line.
    # ----------------------------------------
    plt.axhline(y=0.25, linestyle="--", alpha=0.5)

    # ========================================
    # CPPopt GRAPH
```

```
    # ========================================
    plt.subplot(3,1,3)
    plt.plot(plot_time, plot_cppopt, linewidth=1.5)
    plt.title("Optimal Cerebral Perfusion Pressure (CPPopt)")
    plt.ylabel("CPPopt (mmHg)")
    plt.xlabel("Time")
    plt.grid(True)

    # ----------------------------------------
    # CPPopt physiological limits.
    # ----------------------------------------
    plt.axhline(y=CPP_MIN, linestyle="--", alpha=0.5)
    plt.axhline(y=CPP_MAX, linestyle="--", alpha=0.5)

    # ----------------------------------------
    # Improve spacing.
    # ----------------------------------------
    plt.tight_layout()
    plt.show()

# ========================================
# SECTION 12A : STATUS DISPLAY
# ========================================

def display_monitor_status(
    timestamp,
    map_value,
    icp_value,
    cpp_value,
    prx_value,
    cppopt_value,
    prx_reason,
    cppopt_reason,
    gap_active=False,
    gap_message=""
):
    print("="*70)
    print(f"Time : {timestamp}")
    print("-" * 70)

    # ====================================
    # GAP MODE
    # ====================================
    if gap_active:
        print("MAP     : Data Unavailable")
        print("ICP     : Data Unavailable")
        print("CPP     : Data Unavailable")
```

```
        print("PRx     : Data Unavailable")

        # ---------------------------------
        # During gap:
        # show last valid CPPopt.
        # ---------------------------------
        if not np.isnan(last_valid_cppopt):
            print(f"CPPopt  : {last_valid_cppopt:.2f} (Last Valid Value)")
        else:
            print("CPPopt  : Data Unavailable")

        print("-" * 70)
        print(f"STATUS : {gap_message}")
        print("=" * 70)
        return

    # ======================================
    # NORMAL MODE
    # ======================================
    print(f"MAP     : {map_value:.2f}")
    print(f"ICP     : {icp_value:.2f}")
    print(f"CPP     : {cpp_value:.2f}")

    # -------------------------------------
    # PRx display.
    # -------------------------------------
    if np.isnan(prx_value):
        print("PRx     : Unavailable")
        print(f"Reason  : {prx_reason}")
    else:
        print(f"PRx     : {prx_value:.4f}")

    # -------------------------------------
    # CPPopt display.
    # -------------------------------------
    if np.isnan(cppopt_value):
        print("CPPopt  : Unavailable")
        print(f"Reason  : {cppopt_reason}")
    else:
        print(f"CPPopt  : {cppopt_value:.2f}")

    print("=" * 70)
```

[4]:
```
# ========================================
# SECTION 13 : MAIN LOOP
# ========================================
```

```
while True:
    try:
        # ------------------------------------
        # Read latest CSV.
        # ------------------------------------
        df = pd.read_csv(LIVE_FILE)

        # ------------------------------------
        # CPU optimization.
        # ------------------------------------
        if len(df) <= last_processed_row:
            time.sleep(0.05)
            continue

        # ------------------------------------
        # Process only new rows.
        # ------------------------------------
        new_rows = df.iloc[last_processed_row:]

        for _, row in new_rows.iterrows():
            # ==================================
            # TIMESTAMP
            # ==================================
            timestamp = pd.to_datetime(row["TTDate"] + " " + row["TTTime"])

            # ==================================
            # CURRENT VALUES
            # ==================================
            MAP = row["mean1"]
            ICP = row["mean2"]
            CPP = calculate_cpp(MAP, ICP)

            latest_map = MAP
            latest_icp = ICP
            latest_cpp = CPP

            # ==================================
            # GAP DETECTION
            # ==================================
            gap_found, gap_sec = check_gap(last_timestamp, timestamp)

            # ==================================
            # HANDLE GAP
            # ==================================
            if gap_found:
                # ----------------------------
                # Insert NaN points in graphs.
```

```
                # ------------------------------
                insert_gap_nan_points(last_timestamp, timestamp)

                # ------------------------------
                # Missing samples.
                # ------------------------------
                missing_samples = int(round(gap_sec / REAL_SAMPLE_INTERVAL))

                # ------------------------------
                # Replay wait duration.
                # ------------------------------
                replay_gap_time = missing_samples * REPLAY_INTERVAL
                gap_message = f"Data gap detected ({gap_sec:.1f} sec,␣
↪{missing_samples} samples)"

                # ------------------------------
                # Show unavailable status.
                # ------------------------------
                display_monitor_status(
                    timestamp=timestamp,
                    map_value=np.nan,
                    icp_value=np.nan,
                    cpp_value=np.nan,
                    prx_value=np.nan,
                    cppopt_value=np.nan,
                    prx_reason="Gap detected",
                    cppopt_reason="",
                    gap_active=True,
                    gap_message=gap_message
                )

                # ------------------------------
                # Simulate missing time.
                # ------------------------------
                time.sleep(replay_gap_time)

            # ================================
            # UPDATE BUFFERS
            # ================================
            map_buffer.append(MAP)
            icp_buffer.append(ICP)

            # ================================
            # PRx CALCULATION
            # ================================
            current_prx, prx_reason = calculate_prx(list(map_buffer),␣
↪list(icp_buffer))
```

```
            # ==================================
            # CPP HISTORY
            # ==================================
            if not np.isnan(current_prx):
                mean_cpp = np.mean(np.array(map_buffer) - np.array(icp_buffer))
                prx_history.append(current_prx)
                mean_cpp_history.append(mean_cpp)
                prx_time_history.append(timestamp)

            # ==================================
            # CPPopt CALCULATION
            # ==================================
            current_cppopt, cppopt_reason = calculate_cppopt(prx_history,␣
↪mean_cpp_history)

            # ==================================
            # STORE LAST VALID CPPopt
            # ==================================
            if not np.isnan(current_cppopt):
                last_valid_cppopt = current_cppopt

            # ==================================
            # GRAPH STORAGE
            # ==================================
            time_series.append(timestamp)
            cpp_series.append(CPP)
            prx_series.append(current_prx)

            # ---------------------------------
            # Use latest valid CPPopt
            # when current value unavailable.
            # ---------------------------------
            if np.isnan(current_cppopt):
                cppopt_series.append(last_valid_cppopt)
            else:
                cppopt_series.append(current_cppopt)
                cppopt_time_history.append(timestamp)

            # ==================================
            # UPDATE STATUS
            # ==================================
            last_timestamp = timestamp

            # ==================================
            # LIVE GRAPH
            # ==================================
```

```
            plot_live()

            # ==================================
            # MONITOR DISPLAY
            # ==================================
            display_monitor_status(
                timestamp=timestamp,
                map_value=latest_map,
                icp_value=latest_icp,
                cpp_value=latest_cpp,
                prx_value=current_prx,
                cppopt_value=(current_cppopt if not np.isnan(current_cppopt)␣
↪else last_valid_cppopt),
                prx_reason=prx_reason,
                cppopt_reason=cppopt_reason,
                gap_active=False,
                gap_message=""
            )

        # ======================================
        # UPDATE PROCESSED ROW COUNT
        # ======================================
        last_processed_row = len(df)

    except Exception as e:
        print(f"Error : {str(e)}")
        time.sleep(1)
```

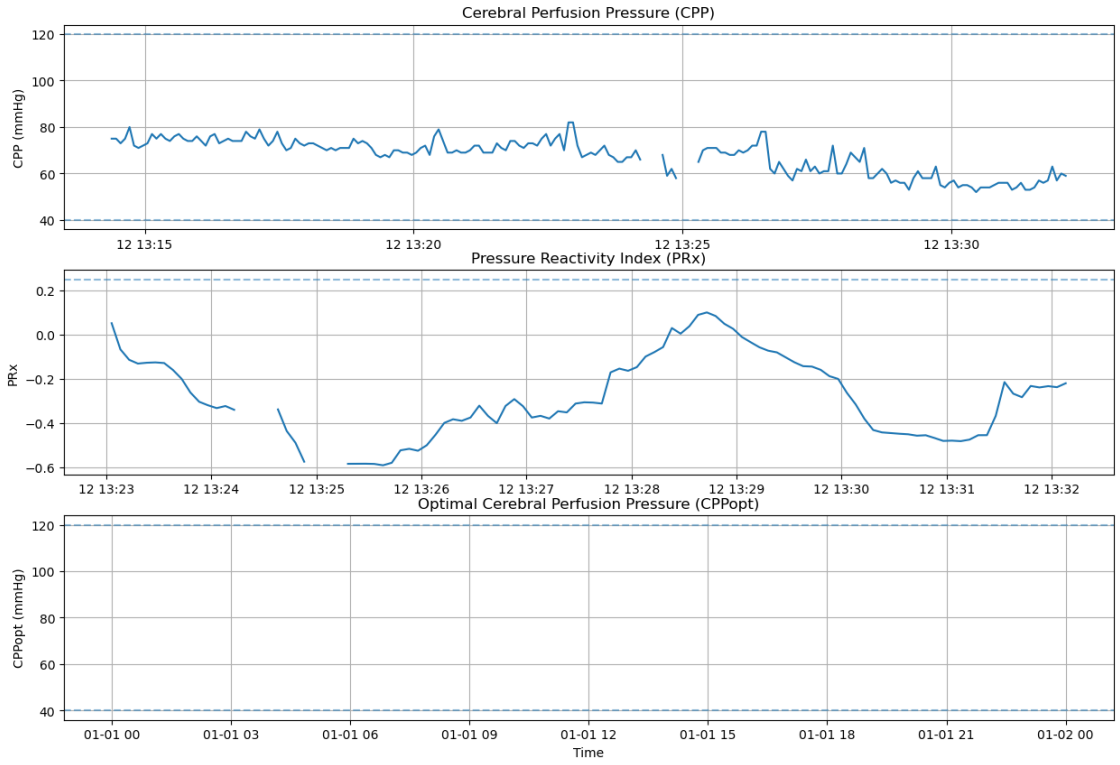
```
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
Cell In[4], line 143
    138 last_timestamp = timestamp
    140 # ==================================
    141 # LIVE GRAPH
    142 # ==================================
--> 143 plot_live()
    145 # ==================================
    146 # MONITOR DISPLAY
    147 # ==================================
    148 display_monitor_status(
    149     timestamp=timestamp,
    150     map_value=latest_map,
   (…)    158     gap_message=""
    159 )

Cell In[3], line 79, in plot_live()
     74 plt.axhline(y=CPP_MAX, linestyle="--", alpha=0.5)
```

```
     76 # ---------------------------------------
     77 # Improve spacing.
     78 # ---------------------------------------
---> 79 plt.tight_layout()
     80 plt.show()

File ~\anaconda3\Lib\site-packages\matplotlib\pyplot.py:2844, in␣
 ↪tight_layout(pad, h_pad, w_pad, rect)
   2836 @_copy_docstring_and_deprecators(Figure.tight_layout)
   2837 def tight_layout(
   2838     *,
   (…)   2842     rect: tuple[float, float, float, float] | None = None,
   2843 ) -> None:
-> 2844     gcf().tight_layout(pad=pad, h_pad=h_pad, w_pad=w_pad, rect=rect)

File ~\anaconda3\Lib\site-packages\matplotlib\figure.py:3640, in Figure.
 ↪tight_layout(self, pad, h_pad, w_pad, rect)
   3638 previous_engine = self.get_layout_engine()
   3639 self.set_layout_engine(engine)
-> 3640 engine.execute(self)
   3641 if previous_engine is not None and not isinstance(
   3642     previous_engine, (TightLayoutEngine, PlaceHolderLayoutEngine)
   3643 ):
   3644     _api.warn_external('The figure layout has changed to tight')

File ~\anaconda3\Lib\site-packages\matplotlib\layout_engine.py:188, in␣
 ↪TightLayoutEngine.execute(self, fig)
    186 renderer = fig._get_renderer()
    187 with getattr(renderer, "_draw_disabled", nullcontext)():
--> 188     kwargs = get_tight_layout_figure(
    189         fig, fig.axes, get_subplotspec_list(fig.axes), renderer,
    190         pad=info[     ], h_pad=info[       ], w_pad=info[       ],
    191         rect=info[      ])
    192 if kwargs:
    193     fig.subplots_adjust(**kwargs)

File ~\anaconda3\Lib\site-packages\matplotlib\_tight_layout.py:266, in␣
 ↪get_tight_layout_figure(fig, axes_list, subplotspec_list, renderer, pad,␣
 ↪h_pad, w_pad, rect)
    261         return {}
    262     span_pairs.append((
    263         slice(ss.rowspan.start * div_row, ss.rowspan.stop * div_row),
    264         slice(ss.colspan.start * div_col, ss.colspan.stop * div_col)))
--> 266 kwargs = _auto_adjust_subplotpars(fig, renderer,
    267                                    shape=(max_nrows, max_ncols),
    268                                    span_pairs=span_pairs,
    269                                    subplot_list=subplot_list,
    270                                    ax_bbox_list=ax_bbox_list,
```

```
    271                                       pad=pad, h_pad=h_pad, w_pad=w_pad)
    273 # kwargs can be none if tight_layout fails…
    274 if rect is not None and kwargs is not None:
    275     # if rect is given, the whole subplots area (including
    276     # labels) will fit into the rect instead of the
   (…)    280     # auto_adjust_subplotpars twice, where the second run
    281     # with adjusted rect parameters.

File ~\anaconda3\Lib\site-packages\matplotlib\_tight_layout.py:82, in␣
 ↪_auto_adjust_subplotpars(fig, renderer, shape, span_pairs, subplot_list,␣
 ↪ax_bbox_list, pad, h_pad, w_pad, rect)
     80 for ax in subplots:
     81     if ax.get_visible():
---> 82         bb += [martist._get_tightbbox_for_layout_only(ax, renderer)]
     84 tight_bbox_raw = Bbox.union(bb)
     85 tight_bbox = fig.transFigure.inverted().transform_bbox(tight_bbox_raw)

File ~\anaconda3\Lib\site-packages\matplotlib\artist.py:1402, in␣
 ↪_get_tightbbox_for_layout_only(obj, *args, **kwargs)
   1396 """
   1397 Matplotlib's `.Axes.get_tightbbox` and `.Axis.get_tightbbox` support a
   1398 *for_layout_only* kwarg; this helper tries to use the kwarg but skips i
   1399 when encountering third-party subclasses that do not support it.
   1400 """
   1401 try:
-> 1402     return␣
 ↪obj.get_tightbbox(*args, **{**kwargs,                    : True})
   1403 except TypeError:
   1404     return obj.get_tightbbox(*args, **kwargs)

File ~\anaconda3\Lib\site-packages\matplotlib\axes\_base.py:4564, in _AxesBase.
 ↪get_tightbbox(self, renderer, call_axes_locator, bbox_extra_artists,␣
 ↪for_layout_only)
   4562 for axis in self._axis_map.values():
   4563     if self.axison and axis.get_visible():
-> 4564         ba = martist._get_tightbbox_for_layout_only(axis, renderer)
   4565         if ba:
   4566             bb.append(ba)

File ~\anaconda3\Lib\site-packages\matplotlib\artist.py:1402, in␣
 ↪_get_tightbbox_for_layout_only(obj, *args, **kwargs)
   1396 """
   1397 Matplotlib's `.Axes.get_tightbbox` and `.Axis.get_tightbbox` support a
   1398 *for_layout_only* kwarg; this helper tries to use the kwarg but skips i
   1399 when encountering third-party subclasses that do not support it.
   1400 """
   1401 try:
```

```
-> 1402     return␣
 ↪obj.get_tightbbox(*args, **{**kwargs,                    : True})
  1403 except TypeError:
  1404     return obj.get_tightbbox(*args, **kwargs)

File ~\anaconda3\Lib\site-packages\matplotlib\axis.py:1356, in Axis.
 ↪get_tightbbox(self, renderer, for_layout_only)
  1353 self._update_label_position(renderer)
  1355 # go back to just this axis's tick labels
-> 1356 tlb1, tlb2 = self._get_ticklabel_bboxes(ticks_to_draw, renderer)
  1358 self._update_offset_text_position(tlb1, tlb2)
  1359 self.offsetText.set_text(self.major.formatter.get_offset())

File ~\anaconda3\Lib\site-packages\matplotlib\axis.py:1332, in Axis.
 ↪_get_ticklabel_bboxes(self, ticks, renderer)
  1330 if renderer is None:
  1331     renderer = self.get_figure(root=True)._get_renderer()
-> 1332 return ([tick.label1.get_window_extent(renderer)
  1333          for tick in ticks if tick.label1.get_visible()],
  1334         [tick.label2.get_window_extent(renderer)
  1335          for tick in ticks if tick.label2.get_visible()])

File ~\anaconda3\Lib\site-packages\matplotlib\text.py:971, in Text.
 ↪get_window_extent(self, renderer, dpi)
   969 bbox, info, descent = self._get_layout(self._renderer)
   970 x, y = self.get_unitless_position()
--> 971 x, y = self.get_transform().transform((x, y))
   972 bbox = bbox.translated(x, y)
   973 return bbox

File ~\anaconda3\Lib\site-packages\matplotlib\transforms.py:1496, in Transform.
 ↪transform(self, values)
  1493 values = values.reshape((-1, self.input_dims))
  1495 # Transform the values
-> 1496 res = self.transform_affine(self.transform_non_affine(values))
  1498 # Convert the result back to the shape of the input values.
  1499 if ndim == 0:

File ~\anaconda3\Lib\site-packages\matplotlib\transforms.py:2411, in␣
 ↪CompositeGenericTransform.transform_affine(self, values)
  2409 def transform_affine(self, values):
  2410     # docstring inherited
-> 2411     return self.get_affine().transform(values)

File ~\anaconda3\Lib\site-packages\matplotlib\transforms.py:2437, in␣
 ↪CompositeGenericTransform.get_affine(self)
  2435     return self._b.get_affine()
  2436 else:
```

```
-> 2437     return Affine2D(np.dot(self._b.get_affine().get_matrix(),
   2438                            self._a.get_affine().get_matrix()))

KeyboardInterrupt:
```



[ ]: